ESCOLA SUPERIOR
DE ENGENHARIA E
TECNOLOGIA

TESP IA
Programação Computadores I
Lists and loops

Marco Tereso - marco.tereso@islasantarem.pt

Lists

- One of the most used objects in Python is the List.
  Syntax:

```
number_list = [1, 2, 3]
print(number_list)
```

```
[1, 2, 3]
```

- The previous list is a list of integers, but it can be of any type e.g.:

```
float_list = [1.2 , 5.2]
string_list = ["Hello", "World", "!"]
matrix_integer_list = [[1,2,3], [1,2,3], [1,2,3]]
mix_list = [1.1, "uhuuu", [5,5,5]]
```

Lists

- A function that we can run on any sequence in Python is len([variable]).
- It shows us the length of the sequence.

```
number_list = [1, 2, 3]
print(len(number_list))
```

```
3
```

- Lists have positions equal to those of Strings, but unlike them, these positions can be changed at any time:

```
number_list = [1,2,3,4]
number_list[0] = 5
print(number_list)
```

```
[5, 2, 3, 4]
```

Lists

- Working with matrices

```
list1 = [1,2,3]
list2 = [6,5,5]
list3 = [8,8,8]
matrix = [list1, list2, list3]
print(matrix)
```

```
[[1, 2, 3], [6, 5, 5], [8, 8, 8]]
```

```
print(matrix[0])
```

```
[1, 2, 3]
```

```
print(matrix[0][1])
```

```
2
```

```
print(list1[1])
```

```
2
```

List Manipulation

- Append()
  - The append function adds a value to the end of the list:

```
number_list = [1,2,3]
print(number_list)
```

```
[1, 2, 3]
```

```
number_list.append(9)
print(number_list)
```

```
[1, 2, 3, 9]
```

```
number_list.append("oi")
print(number_list)
```

```
[1, 2, 3, 9, 'oi']
```

List Manipulation

- Insert()
  - Unlike append, which inserts only at the end of the list, with insert you can insert anywhere by passing the index and value as parameters.

```
number_list = [1,2,3]
number_list.insert(0,9)
print(number_list)
```

```
[9, 1, 2, 3]
```

```
number_list.insert(0,"ola Mundo!")
print(number_list)
```

```
['ola Mundo!', 9, 1, 2, 3]
```

List Manipulation

- Remove()
  - Passing as a parameter the value you want to remove

```
number_list = [1,2,3]
number_list.remove(2)
print(number_list)
```

```
[1, 3]
```

```
number_list.remove(9)
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
Cell In[17], line 1
----> 1 number_list.remove(9)

ValueError: list.remove(x): x not in list
```

List Manipulation

- pop()
  - It removes and returns the last value from the list. For those who know stack and had difficulty implementing the push and pop methods, know that in Python they are already implemented! push=append, pop=pop

```
number_list = [1,2,3]
print(number_list.pop())
```

```
3
```

```
print(number_list.pop())
```

```
2
```

```
print(number_list)
```

```
[1]
```

Tuples

- Tuples are objects similar to lists, except that tuples are immutable. Once created, they cannot be modified.

```
t = (1,2,3)
print(t)
```

```
(1, 2, 3)
```

```
t[0] = 5
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[25], line 1
----> 1 t[0] = 5

TypeError: 'tuple' object does not support item assignment
```

```
print(t[0])
```

```
1
```

Tuples

- With tuples we can perform different manipulation of variables. This technique is called "packing-unpacking".

```
(a,b) = ("ola", 5.734)
print(a)
print(b)
```

```
ola
5.734
```

```
a,b = "ola", 5.734
print(a)
print(b)
```

```
ola
5.734
```

Tuples

- With tuples we can perform different manipulation of variables. This technique is called "packing-unpacking".

```
(a,b) = ("hello", 5.734)
print(a)
print(b)
```

```
hello
5.734
```

```
a,b = "hello", 5.734
print(a)
print(b)
```

```
hello
5.734
```

- The above code assigned "hello" to a and 5.734 to b.

Tuples

- We can easily swap values between two variables:

```
a = 5
b = 1
a,b = b,a
print(a)
print(b)
```

```
1
5
```

Dictionaries

- Dictionaries are not sequences like strings and lists. Their form of indexing is different, using keys to access values

```
d = {"one":"value 1",2:"twooo", "name":"Peter"}
print(d)
```

```
{'one': 'value 1', 2: 'twooo', 'name': 'Peter'}
```

```
print(d["one"])
```

```
value 1
```

```
print(d["name"])
```

```
Peter
```

```
print(d[2])
```

```
twooo
```

```
print(d["one"]+" have "+str(d[2])+" "+d["name"])
```

```
value 1 have twooo Peter
```

```
d["name"] = "Somebody"
print(d["name"])
```

```
Somebody
```

Dictionaries

- To find out what the keys of a dictionary are, we use the keys() method of the dictionary object. We can also check if the dictionary has any keys using the has_key([key]) method:

```
d = {"one":"value 1",2:"twooo", "name":"Peter","animal":"pig" }
print(d.keys())
```

```
dict_keys(['one', 2, 'name', 'animal'])
```

Loop - while

- Loop structures allow you to repeat a section of code until the condition passed is met. The "While" structure is as follows:

While [condition]:

instructions

- Note: don't forget to increment!! -> var += 1

Loop - while

- While the condition is true the code after the " : (colon)" is repeated N times.
- Example:

```
a = 0
while a < 5:
    print(a)
    a = a + 1
```

```
0
1
2
3
4
```

Loop - for

- Unlike other programming languages, the For loop in Python works differently.
- Instead of repeating until its condition is reached, as in While, the For loop runs through Sequence Structures (String, List and Tuples) and its stopping condition is the length of the sequence.
- Imagine the sequence: [1,2,1,8] In this case, the for loop would repeat the code in its block *4* times.

Loop - for

- The structure of FOR:

for [variable] in [variable sequence] :

instructions

```
list1 = [1,2,1,8]
for i in list1:
    print("value: "+str(i))
```

```
value: 1
value: 2
value: 1
value: 8
```

Loop - for

- In the example below, the variable " i " loops through the sequential variable taking the value of each position and printing it.

```
s = "Hello"
for i in s:
    print("value: "+str(i))
```

```
value: H
value: e
value: l
value: l
value: o
```

Loop - for

- For also traverses the Dictionary structure, but in a different way than the others.
- The variable receives the value of the dictionary key
- **Example:**

```
a = {22:"uhu", 123:"xx", 57:5777}
for t in a:
    print(t)
```

```
22
123
57
```

Loop - for

- There is a function that creates a list from the value "0" to the value that was passed as a parameter. This function is " range() "

```
list1 = range(5)
print(list1)
for aux in range(5):
    print(aux)
```

```
range(0, 5)
0
1
2
3
4
```

Break and exit

- The Break command forces the interruption of the loop, without having to wait for the condition

```
a = 0
while a < 5:
    print("entered while")
    break
    print("went through the break")
```

```
entered while
```

- The " exit() " function ends the application.